

Università degli Studi di Napoli Federico II

Corso di Laurea in Informatica



---

# Conquista del territorio

Elaborato tecnico per il corso di Laboratorio di Sistemi Operativi

---

Docente: Prof. Alberto Finzi

Lavoro a cura di:

Giulio Pacella - Matricola N86005257

Christian Petrone - Matricola N86005370

Anno Accademico 2025/2026

# Indice

|          |                                                                       |           |
|----------|-----------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduzione al gioco</b>                                          | <b>3</b>  |
| 1.1      | Parametri principali . . . . .                                        | 3         |
| <b>2</b> | <b>Architettura del sistema</b>                                       | <b>4</b>  |
| 2.1      | Il server . . . . .                                                   | 4         |
| 2.2      | Il client . . . . .                                                   | 4         |
| 2.3      | Organizzazione modulare . . . . .                                     | 4         |
| <b>3</b> | <b>Il protocollo applicativo</b>                                      | <b>5</b>  |
| 3.1      | Comandi Client → Server . . . . .                                     | 5         |
| 3.2      | Risposte Server → Client . . . . .                                    | 5         |
| 3.3      | Simboli delle celle . . . . .                                         | 5         |
| 3.4      | Serializzazione (formato esatto) . . . . .                            | 5         |
| 3.5      | Esempio di sessione reale . . . . .                                   | 6         |
| 3.6      | Framing su TCP . . . . .                                              | 6         |
| <b>4</b> | <b>Scelte di implementazione</b>                                      | <b>7</b>  |
| 4.1      | Perché <code>select()</code> e non <code>fork/thread</code> . . . . . | 7         |
| 4.2      | Il tetto ai client e <code>FD_SETSIZE</code> . . . . .                | 8         |
| 4.3      | Il loop del server . . . . .                                          | 8         |
| 4.4      | Il loop del client . . . . .                                          | 8         |
| <b>5</b> | <b>Dettagli interessanti</b>                                          | <b>10</b> |
| 5.1      | I/O non bloccante: framing e coda di output . . . . .                 | 10        |
| 5.2      | Fog-of-war e "luce globale" . . . . .                                 | 10        |
| 5.3      | Conquista e ribaltamento dei punteggi . . . . .                       | 10        |
| 5.4      | Ciclo di vita della partita . . . . .                                 | 10        |
| 5.5      | Macchina a stati e robustezza . . . . .                               | 10        |
| 5.6      | Gestione dei segnali . . . . .                                        | 11        |
| <b>6</b> | <b>Guida alla compilazione e all'uso</b>                              | <b>12</b> |
| 6.1      | Compilazione . . . . .                                                | 12        |
| 6.2      | Avvio del server . . . . .                                            | 12        |
| 6.3      | Avvio del client . . . . .                                            | 12        |
| 6.4      | Verifica della memoria . . . . .                                      | 12        |
| 6.5      | Esecuzione in container Docker . . . . .                              | 13        |
| <b>7</b> | <b>Limiti noti ed estensioni future</b>                               | <b>14</b> |
| 7.1      | Limiti noti . . . . .                                                 | 14        |

|     |                                       |    |
|-----|---------------------------------------|----|
| 7.2 | Possibili estensioni future . . . . . | 14 |
|-----|---------------------------------------|----|

# 1 Introduzione al gioco

*Conquista del territorio* è un gioco multigiocatore in tempo reale in cui più giocatori si muovono su una griglia bidimensionale con l'obiettivo di conquistare il maggior numero di celle. Ogni giocatore controlla un personaggio che occupa una cella della mappa; a ogni spostamento la cella di arrivo diventa di sua proprietà. Se la cella apparteneva a un avversario, gli viene rubata. Al termine del tempo di partita vince chi possiede più celle.

La mappa è una griglia di  $20 \times 20$  celle contenente alcuni muri generati casualmente (densità  $\sim 20\%$ ), che bloccano il passaggio. Un elemento centrale del gioco è la visibilità limitata (fog-of-war): ogni giocatore vede solo una zona  $5 \times 5$  centrata sulla propria posizione, che abbiamo immaginato essere come una torcia che lo illumina. I muri, una volta scoperti, restano ricordati; le proprietà delle celle e le posizioni degli avversari, invece, si vedono solo dentro la torcia. A intervalli regolari il server invia una panoramica globale che, per pochi secondi, "accende la luce" e mostra a tutti lo stato dell'intera mappa (fuorché i muri non ancora scoperti).

## 1.1 Parametri principali

Tutti i parametri di gioco sono raccolti in `common.h` e sono modificabili in un unico punto, ciò rende il gioco potenzialmente personalizzabile in modo semplice per rendere ogni partita diversa dalle altre.

| Parametro                | Valore         | Significato                                       |
|--------------------------|----------------|---------------------------------------------------|
| MAPPA_H $\times$ MAPPA_W | $20 \times 20$ | dimensioni della griglia                          |
| R_FOG                    | 2              | raggio torcia $\rightarrow$ finestra $5 \times 5$ |
| DENSITA_MURI             | 20             | percentuale di muri                               |
| T_BROADCAST              | 20 s           | intervallo tra due panoramiche globali            |
| T_FLASH                  | 2 s            | durata della "luce globale"                       |
| T_PARTITA                | 100 s          | durata massima di una partita                     |
| T_PAUSA                  | 10 s           | durata della pausa dopo un gameover               |
| MAX_CLIENT               | 1000           | tetto ai client simultanei                        |

## 2 Architettura del sistema

Il progetto segue un'architettura client/server: due programmi distinti, eseguiti come processi separati (su macchine diverse o in locale), che comunicano attraverso la rete via socket TCP. I ruoli sono nettamente asimmetrici.

### 2.1 Il server

Il server è autorevole: tutta la logica di gioco risiede esclusivamente al suo interno. È l'unico a conoscere la mappa completa, a decidere l'esito di ogni mossa, a calcolare i punteggi e a scandire il tempo di partita. In particolare:

- accetta le connessioni e gestisce il ciclo di vita di ogni client;
- autentica gli utenti (registrazione e login) e ne conserva gli account;
- valida ogni comando rispetto alle regole e allo stato del giocatore;
- aggiorna lo stato del mondo (posizioni, proprietà delle celle, punteggi);
- invia a ciascun client la propria visuale (torcia) e, periodicamente, la panoramica globale.

Ciò permette due grandi vantaggi: eliminare gli imbrogli (il client non può alterare lo stato, può solo chiederne la modifica) e azzerare le race condition, perché un unico processo elabora un comando per volta senza stato condiviso tra flussi concorrenti.

### 2.2 Il client

Il client è volutamente "leggero": non conosce né controlla le regole del gioco. Si connette al server, gli inoltra i comandi digitati dall'utente, riceve i messaggi e disegna a schermo la mappa in ASCII. Mantiene soltanto un modello locale di ciò che ha già visto (muri scoperti, proprietà, posizioni), al solo scopo di disegnare una vista coerente tra gli aggiornamenti. Ogni decisione di gioco resta al server.

### 2.3 Organizzazione modulare

Il codice è suddiviso in moduli a responsabilità singola:

| Modulo                     | Lato   | Responsabilità                                            |
|----------------------------|--------|-----------------------------------------------------------|
| <code>common.h</code>      | comune | parametri, limiti, feature-test macro, enum di stato      |
| <code>protocol.h</code>    | comune | costanti del protocollo, simboli delle mappe              |
| <code>net_util.[ch]</code> | comune | socket, buffer di input, coda di output                   |
| <code>users.[ch]</code>    | server | registrazione/verifica account, persistenza               |
| <code>game.[ch]</code>     | server | mappa, mosse, fog-of-war, punteggi, serializzazione       |
| <code>server.c</code>      | server | loop <code>select()</code> , dispatch, broadcast, segnali |
| <code>client.c</code>      | client | UI non bloccante e rendering ASCII/colori                 |

I moduli `net_util`, `common.h` e `protocol.h` sono condivisi: definiscono il formato dei messaggi e le primitive di rete comuni che client e server devono rispettare in modo identico.

Inoltre `common.h` definisce le feature-test macro (`_POSIX_C_SOURCE`, `_DEFAULT_SOURCE`) dunque deve essere il primo `#include` di ogni `.c`, prima di qualunque header di sistema. In questo modo funzioni come `getaddrinfo()`, `fcntl()`, `snprintf()` e la costante `MSG_NOSIGNAL` risultano dichiarate correttamente e il progetto compila senza alcun warning con `-Wall -Wextra -pedantic -std=c11`, che altrimenti bloccherebbero la compilazione non riconoscendo le firme.

### 3 Il protocollo applicativo

Il protocollo è testuale e orientato sulle linee: ogni messaggio, in entrambe le direzioni, è una riga terminata da `\n`. Questa scelta lo rende leggibile a occhio, facile da debuggare e testabile a mano con `nc` (netcat).

#### 3.1 Comandi Client → Server

|                        |                                               |
|------------------------|-----------------------------------------------|
| REGISTER <nick> <pass> | registra un nuovo account (nick libero)       |
| LOGIN <nick> <pass>    | autentica ed entra in gioco                   |
| MOVE <U D L R>         | muove di una cella (Up/Down/Left/Right)       |
| WHO                    | elenco dei giocatori loggati (risposta USERS) |
| QUIT                   | chiude la sessione                            |

#### 3.2 Risposte Server → Client

|              |                                          |                              |
|--------------|------------------------------------------|------------------------------|
| OK [msg]     | esito positivo (con messaggio opzionale) |                              |
| ERR <msg>    | esito negativo, con spiegazione          |                              |
| LOCAL ...    | mappa locale (torcia)                    | -- multi-riga, chiusa da END |
| GLOBAL ...   | mappa globale (proprietà)                | -- multi-riga, chiusa da END |
| USERS ...    | elenco giocatori                         | -- multi-riga, chiuso da END |
| GAMEOVER ... | classifica di fine partita               | -- multi-riga, chiusa da END |

#### 3.3 Simboli delle celle

- # muro scoperto
- . cella libera
- 0-9/a-z cella posseduta dal giocatore con quell'id
- @ posizione propria (solo LOCAL)
- ? fuori mappa / ignoto.

#### 3.4 Serializzazione (formato esatto)

Il formato è fissato e deve essere replicato identico tra `game.c` (produzione) e `client.c` (parsing):

|                                 |                                         |
|---------------------------------|-----------------------------------------|
| LOCAL <cx> <cy> <r>\n           | centro = posizione del giocatore        |
| <2r+1 righe da 2r+1 caratteri>  | finestra centrata su (cx,cy)            |
| END\n                           |                                         |
| GLOBAL <W> <H> <nplayers>\n     |                                         |
| <H righe da W caratteri>        | mappa proprietà, senza '@' e senza muri |
| P <id> <nick> <x> <y> <score>\n | ripetuta nplayers volte                 |
| END\n                           |                                         |
| USERS <n>\n                     |                                         |
| <id> <nick>\n                   | ripetuta n volte                        |
| END\n                           |                                         |
| GAMEOVER <n>\n                  |                                         |
| <pos> <nick> <score>\n          | classifica decrescente, n righe         |
| END\n                           |                                         |

### 3.5 Esempio di sessione reale

```
C: REGISTER mario segreto
S: OK registrato
C: LOGIN mario segreto
S: OK login id=0
S: GLOBAL 20 20 1
    .....
    ...(altre righe)....
    P 0 mario 12 7 1
    END
C: MOVE U
S: LOCAL 12 6 2
    ..#..
    .....
    ..@..
    ..0..      <- '0' = cella lasciata, ora di mario
    ..#...
    END
C: MOVE R
S: ERR muro      <- il muro entra nel fog e comparira' nei LOCAL successivi
C: WHO
S: USERS 1
    0 mario
    END
```

### 3.6 Framing su TCP

TCP è un flusso di byte, dunque una `read()` può spezzare un messaggio o aggregarne più di uno. Per ovviare a questo problema entrambi i lati accumulano i byte in un `BufferIn` ed estraggono una riga completa per volta cercando il `\n` (tramite `buf_in_estrai_riga`, che tollera anche `\r\n`). Una riga più lunga di `MAX_LINE` (1024 byte) viene scartata con `ERR`, a protezione da input malevolo.

## 4 Scelte di implementazione

### 4.1 Perché `select()` e non `fork/thread`

Il server deve servire molti client contemporaneamente. Ma il problema principale è che una `read()` bloccante su un client congelerebbe il processo ignorando tutti gli altri. Per ovviare a questo problema abbiamo valutato tre approcci:

- **un processo per client (`fork`):** appena arriva un client il server duplica il processo e il figlio si occupa solo di lui, in questo modo anche in presenza di una `read()` bloccante gli altri client non ne risentirebbero. E' un approccio semplice, ma i processi non avrebbero condiviso la memoria, quindi avremmo dovuto mettere mappa e proprietà in una memoria condivisa, con sincronizzazione esplicita.
- **un thread per client (`pthread`):** appena arriva un client il server crea un thread che lo gestisce. In questo caso, i thread condividono la memoria ma potrebbero modificare il suo contenuto nello stesso istante, causando comportamenti anomali del programma. Avremmo dovuto gestire il tutto con dei mutex presenti ad ogni accesso alla memoria condivisa, rischiando anche deadlock.
- **`select()` — la scelta adottata:** un solo processo aspetta su tutti i socket insieme e viene risvegliato solo su quelli pronti. Questo approccio garantisce quattro vantaggi concreti:
  1. **nessuna race condition:** c'è un unico flusso di esecuzione, quindi mentre elaboro la mossa di un giocatore nessun altro tocca lo stato: niente mutex, niente IPC;
  2. **stato condiviso "gratis":** mappa e giocatori sono normali variabili del processo (`Mappa partita` e `Giocatore giocatori[]`);
  3. **timer integrato:** il `timeout` di `select()` fornisce il "battito" per il broadcast periodico e per il controllo del timeout di partita, senza thread timer dedicati;
  4. **scalabilità sufficiente:** un solo processo, nessun overhead di creazione/scheduling di processi o thread.

Lo svantaggio teorico (un client lentissimo potrebbe rallentare gli altri) è neutralizzato rendendo tutti i socket non bloccanti e accodando l'output, in questo modo nessuna operazione di I/O blocca mai il loop (vedi §5.1).

## 4.2 Il tetto ai client e FD\_SETSIZE

L'unico limite di `select()` è di natura didattica. La traccia infatti chiede "nessun limite a priori" al numero di utenti. Un numero davvero illimitato però non è ottenibile con `select()`: ogni file descriptor deve infatti restare sotto `FD_SETSIZE` (tipicamente 1024), perché `FD_SET` su un `fd`  $\geq$  `FD_SETSIZE` è *undefined behavior*. Il limite non è quindi arbitrario ma imposto dal meccanismo di multiplexing. Abbiamo portato `MAX_CLIENT` vicino a quel tetto (1000), lasciando margine per gli `fd` riservati.

A ciò si aggiunge una considerazione di natura ludica che rende il limite del tutto ininfluenza nella pratica. La mappa di gioco è una griglia  $20 \times 20$ , per un totale di 400 celle (ulteriormente ridotte dalla presenza degli ostacoli). Un numero di giocatori dello stesso ordine di grandezza — per non parlare del tetto di 1000 — svuoterebbe completamente di significato la meccanica di conquista: non vi sarebbero celle libere a sufficienza nemmeno per collocare i giocatori nelle rispettive posizioni iniziali, e la conquista del territorio si ridurrebbe a un continuo e istantaneo ribaltamento delle celle. Il gioco è pensato per un numero di partecipanti ragionevolmente contenuto rispetto alla dimensione della mappa; il limite tecnico di 1000 client è pertanto largamente superiore a qualunque soglia di utilizzo sensata, e la sua esistenza non rappresenta in alcun modo una restrizione percepibile dall'utente.

## 4.3 Il loop del server

```
while (!fermati) {
    /* i set vanno RICOSTRUITI a ogni giro: select() li modifica */
    costruisci readfds (listen + tutti i client);
    costruisci writefds (SOLO i client con coda di output non vuota);
    timeout = min(tempo al prossimo broadcast, tempo a fine partita);
    select(maxfd+1, &readfds, &writefds, NULL, &timeout);

    /* compiti periodici guidati dal tempo (timestamp, non solo su timeout) */
    if (partita scaduta) termina_partita(); /* GAMEOVER + azzeramento */
    else if (è ora) broadcast_global();

    if (listening pronto) accetta_conessioni();
    per ogni client pronto in lettura servi_lettura(); /* read+framing+dispatch */
    per ogni client pronto in scrittura coda_drena(); /* svuota la coda */
}
chiudi_tutto(); /* close di tutti i socket + free di tutte le code */
```

Due dettagli importanti. Primo: un client entra in `writefds` **solo** se ha byte pendenti in coda; è essenziale durante un broadcast verso molti client, perché si scrive solo quando il socket è effettivamente scrivibile. Secondo: il `timeout` è "quanto manca al prossimo evento temporale", così il broadcast resta regolare anche con molto traffico.

## 4.4 Il loop del client

Anche il client usa `select()`, ma per un motivo diverso: deve sorvegliare **due sorgenti** contemporaneamente, la **tastiera** (`stdin`) e il **socket** verso il server. Senza multiplexing, restare bloccato su una `read()` da tastiera impedirebbe di ricevere i broadcast del server (e viceversa). Con `select()` l'utente può digitare i comandi mentre gli aggiornamenti arrivano, senza mai bloccarsi.

```

while (attivo) {
    /* sorveglio DUE fd: la tastiera e il socket del server */
    FD_SET(STDIN_FILENO, &rfd);
    FD_SET(fd, &rfd);

    /* timeout impostato SOLO se la "luce globale" e' accesa: allo scadere
       si ridisegna la mappa tornando alla vista-torcia */
    ptv = (luce_globale_accesa) ? &tempo_residuo : NULL;
    select(maxfd+1, &rfd, NULL, NULL, ptv);

    if (dati dal server pronti) {
        read(fd, ...);          /* accumulo nel buffer di input */
        per ogni riga completa estratta:
            processa_server(riga); /* parsing OK/ERR/LOCAL/GLOBAL/... */
    }
    if (input da tastiera pronto) {
        read(STDIN_FILENO, ...); /* stesso framing a righe */
        per ogni riga completa estratta:
            gestisci_input(fd, riga); /* traduce W/A/S/D e invia il comando */
    }
}
close(fd);

```

Alcuni dettagli che distinguono il client dal server:

- **Framing in ingresso:** i dati dal server vengono bufferizzati ed estratti una riga per volta, esattamente come fa il server. I messaggi multi-riga (LOCAL/GLOBAL/USERS/GAMEOVER) vengono accumulati fino alla riga END e solo allora resi a schermo.
- **Output diretto:** il client invia i comandi con `send()` diretto (con ciclo su EINTR e sulle scritture parziali), lasciando il socket in modalità bloccante: a differenza del server, `connetti_a` non chiama `imposta_nonblocking`. La scelta è deliberata. Su un socket bloccante `send()` si ferma soltanto se il buffer di trasmissione del kernel è pieno; poiché ogni comando è di pochi byte, riempirlo richiederebbe migliaia di comandi non ancora letti dal server, condizione irraggiungibile con un utente che digita. Rendere il socket non bloccante imporrebbe quindi una `CodaOut` e un insieme `writelfds` che non avrebbero mai byte da drenare: apparato senza un problema da risolvere. Sul server la stessa analisi porta al risultato opposto.
- **Il timeout finalizzato:** normalmente `select()` attende senza scadenza (NULL); solo mentre la luce globale è accesa viene impostato un timeout pari al tempo che le resta, così allo scadere il loop si risveglia da solo e ridisegna la mappa in vista-torcia.
- **Scorciatoie di movimento:** `gestisci_input` traduce i singoli tasti WASD in MOVE U/L/D/R; ogni altra riga viene inoltrata al server così com'è.

## 5 Dettagli interessanti

### 5.1 I/O non bloccante: framing e coda di output

Con socket non bloccanti anche `send()` può scrivere parzialmente o restituire `EAGAIN`. Per questo non viene mai chiamato `send()` sparso nel codice: ogni client ha una `CodaOut` dinamica. Le risposte vengono **accodate** (`coda_accoda`) e drenate quando il socket è scrivibile (`coda_drena`, che usa `send(..., MSG_NOSIGNAL)`). Se l'invio è parziale o si ferma su `EAGAIN`, il resto rimane in coda e si riprova al giro successivo; il fd viene tolto da `writelfds` quando la coda si svuota. È l'unica memoria dinamica del server, liberata alla disconnessione e nello shutdown: il programma è valgrind-clean.

### 5.2 Fog-of-war e "luce globale"

I muri sono privati: ogni giocatore ne tiene una matrice `scoperto[][]`. A ogni mossa (e allo spawn) `gioco_rivela_fog` marca i muri della finestra  $5 \times 5$ ; le scoperte si accumulano per tutta la partita. Le proprietà delle celle, invece, sono pubbliche nel protocollo: è il client a filtrarle, mostrandole solo dentro la torcia. L'eccezione è la "luce globale": nei `T_FLASH` secondi dopo ogni `GLOBAL`, il client mostra tutto tranne i muri non ancora scoperti. La torcia è delimitata da una cornice: le celle a distanza di  $R\_FOG + 1$  vengono disegnate con `+ - |`.

### 5.3 Conquista e ribaltamento dei punteggi

Entrando in una cella libera, `proprietà[y][x]` viene impostata all'id del giocatore; se la cella era di un altro, cambia padrone. I punteggi non sono mantenuti incrementalmente ma ricalcolati scandendo la mappa prima di ogni `GLOBAL/GAMEOVER`: così restano sempre coerenti anche dopo i ribaltamenti, senza rischio di contatori sfasati.

### 5.4 Ciclo di vita della partita

Il timer globale parte al primo login. Allo scadere di `T_PARTITA` il server invia `GAMEOVER` con la classifica a tutti i loggati, poi azzera lo stato (mappa, proprietà, posizioni, fog) tornando in `PARTITA_FERMA`. I client restano connessi e loggati e una nuova partita riparte automaticamente al primo `MOVE/LOGIN` successivo, senza riavviare nulla.

### 5.5 Macchina a stati e robustezza

Il server applica una piccola macchina a stati: in `ST_CONNESSO` ammette solo `REGISTER/LOGIN/QUIT`; in `ST_IN_GIOCO` ammette `MOVE/WHO/QUIT`. Un comando valido ma nello stato sbagliato produce un `ERR` esplicativo.

## 5.6 Gestione dei segnali

Un server che sta in ascolto per molto tempo deve tenere conto dei segnali, cioè di eventi asincroni che il sistema operativo gli manda e che possono ucciderlo se non vengono gestiti. Nel nostro caso i segnali da trattare sono due.

**SIGPIPE.** Se proviamo a scrivere su un socket che il client ha già chiuso, il sistema ci invia **SIGPIPE**, che per default termina il processo. Basterebbe dunque un giocatore che chiude la finestra del terminale per far cadere il server e con lui la partita di tutti gli altri. Per evitarlo lo ignoriamo con `signal(SIGPIPE, SIG_IGN)`. Così la `send()` non uccide più il processo ma fallisce restituendo l'errore **EPIPE**, che noi trattiamo come una normale disconnessione del client. Per sicurezza usiamo anche il flag `MSG_NOSIGNAL` in tutte le `send()`, che evita del tutto la generazione del segnale.

**SIGINT e SIGTERM.** Sono i segnali di chiusura, il primo generato da Ctrl+C e il secondo dal comando `kill`. Non vogliamo che il server muoia di colpo, perché resterebbero socket aperti e memoria allocata non liberata. Li registriamo quindi con `sigaction()` su un gestore che si limita ad alzare una variabile flag:

```
static volatile sig_atomic_t fermati = 0;

static void on_segnaile(int s) {
    (void)s;          /* non ci interessa quale segnale sia arrivato */
    fermati = 1;
}
```

Il lavoro vero lo fa quindi il ciclo principale, che è un `while (!fermati)` e alla prossima iterazione esce da solo.

Il tipo della deve essere necessariamente `volatile` perché altrimenti il compilatore, vedendo che dentro il ciclo nessuno modifica `fermati`, potrebbe ottimizzare leggendo una volta sola e il ciclo non finirebbe mai. Serve `sig_atomic_t` perché garantisce che la scrittura avvenga in un colpo solo, senza il rischio che il segnale arrivi a metà dell'assegnamento.

C'è infine un dettaglio pratico: se il segnale arriva mentre il processo è fermo dentro la `select()`, questa termina subito restituendo `-1` con `errno` uguale a `EINTR`. Non è un vero errore, quindi non lo trattiamo come tale: semplicemente rifacciamo la `select()`, e se nel frattempo il flag è stato alzato usciamo dal ciclo.

Una volta usciti chiamiamo `chiudi_tutto()`, che per ogni client ancora collegato chiama `rimuovi_client()` (chiude il socket e libera la sua `CodaOut` con `coda_free()`) e alla fine chiude il socket di ascolto. In questo modo il server resta valgrind-clean anche quando lo fermiamo con Ctrl+C, e non solo quando i client si disconnettono uno alla volta.

## 6 Guida alla compilazione e all'uso

### 6.1 Compilazione

Il progetto usa un Makefile. I flag di compilazione sono già impostati (-Wall -Wextra -pedantic -std=c11 -O2):

```
make           # compila server e client
make server    # solo il server
make client    # solo il client
make clean     # rimuove eseguibili e file oggetto
```

### 6.2 Avvio del server

```
./server <porta> [seed]
./server 8080          # porta 8080, mappa casuale
./server 8080 42       # seed fisso => mappa riproducibile (utile in test)
```

Il server non scrive su `stdout` e non legge da `stdin`; usa `stderr` solo per errori fatali di avvio. Il seed opzionale rende la mappa deterministica.

### 6.3 Avvio del client

```
./client <host> <porta>
./client 127.0.0.1 8080
```

Il client accetta hostname o indirizzo IP (via `getaddrinfo`). Una volta connesso si digitano i comandi del protocollo; per il movimento sono disponibili le scorciatoie `W A S D` (tradotte in `MOVE U/L/D/R`). I colori ANSI si attivano solo se `stdout` è un terminale.

### 6.4 Verifica della memoria

```
valgrind --leak-check=full ./server 8080 # deve restare "clean"
```

## 6.5 Esecuzione in container Docker

Il progetto usa esclusivamente system call UNIX (`select()`, socket BSD, `sigaction`), quindi richiede un ambiente POSIX: un container permette di eseguirlo su qualunque host — Linux, macOS o Windows — senza installare la toolchain e senza dipendere dalla versione di libc o di gcc presente sulla macchina. Su macOS e Windows il container gira comunque su un kernel Linux, fornito dalla macchina virtuale di Docker Desktop.

**Build multi-stage:** Il `Dockerfile` è organizzato in tre stadi. Il primo (`build`) installa `build-essential` e compila con lo stesso `Makefile` del progetto, con i flag invariati. Gli altri due (`client` e `server`) ripartono da un'immagine Debian pulita e vi copiano soltanto l'eseguibile prodotto. Il compilatore serve alla costruzione, non all'esecuzione: l'immagine finale non lo contiene, pesa circa 114MB e riduce la superficie d'attacco. Client e server condividono lo stadio di compilazione, che viene quindi eseguito una volta sola per entrambe le immagini.

**Uso:** Il file `docker-compose.yml` descrive i due servizi, la rete privata che li collega e il volume dei dati:

```
docker compose build          # costruisce le due immagini
docker compose up -d server   # avvia il server in background
docker compose run --rm client # apre un client interattivo
docker compose logs -f server  # segue lo stderr del server
docker compose down           # ferma tutto (-v cancella anche gli account)
```

Compose registra il container del server sulla rete interna con l'hostname `server`: il client vi si connette per nome, senza indirizzi IP espliciti. Lanciando `docker compose run` in più terminali si ottengono più giocatori sulla stessa partita. La porta è comunque pubblicata sull'host, quindi resta possibile usare il client nativo con `./client 127.0.0.1 8080`.

**Persistenza degli account:** Il filesystem di un container è effimero: alla rimozione, tutto ciò che vi è stato scritto viene perso. Poiché `users.c` apre `users.dat` con un percorso relativo, il file nasce nella directory di lavoro; questa è fissata a `/data`, su cui viene montato un volume persistente. La separazione da `/app`, che contiene l'eseguibile, non è casuale: montare il volume sulla directory del binario ne farebbe persistere una copia nel volume stesso, che continuerebbe a essere eseguita anche dopo una ricostruzione dell'immagine.

## 7 Limiti noti ed estensioni future

### 7.1 Limiti noti

- **Password in chiaro:** gli account sono salvati su `users.dat` nel formato `nick pass` per riga, senza cifratura né hashing.
- **Id di slot ed eredità delle celle:** l'id del giocatore coincide con l'indice di slot. Alla disconnessione le celle conquistate restano sulla mappa con quell'id; se un nuovo client riusa lo stesso slot le eredita. Lo stato viene comunque azzerato a fine partita.
- **Tetto a FD\_SETSIZE:** come discusso, `select()` non consente un numero realmente illimitato di client.
- **Una sola partita globale:** tutti i giocatori condividono la stessa mappa; non esistono "stanze" separate.

### 7.2 Possibili estensioni future

- **Sicurezza degli account:** sostituire le password in chiaro con un hash salato (es. tramite `crypt()`), e opzionalmente cifrare il canale con TLS.
- **Stanze multiple:** gestire più partite indipendenti in parallelo, con un comando di `JOIN/CREATE` della stanza.
- **Riconnessione e sessioni:** disaccoppiare l'identità del giocatore dallo slot (token di sessione) per permettere di rientrare mantenendo le proprie celle.
- **Nuove meccaniche di gioco:** bonus temporanei, ostacoli mobili, celle speciali, oppure una modalità a squadre.
- **Interfaccia grafica:** un client con GUI o web che parli lo stesso protocollo testuale, senza toccare il server.